

Practical No:1

1. Design and implement Parallel Breadth First Search and Depth First Search based on existing algorithms using OpenMP. Use a Tree or an undirected graph for BFS and DFS .

Code:

```
#include <iostream>
#include <cstring>
#include <omp.h>
using namespace std;
int graph[20][20], visited[20], nodes;
// BFS
void bfs(int start) {
    int queue[20], nextQueue[20];
    int size = 1, nextSize = 0;
    memset(visited, 0, sizeof(visited));
    queue[0] = start;
    visited[start] = 1;
    cout << "BFS: ";
    while (size) {
        nextSize = 0;
        #pragma omp parallel for
        for (int i = 0; i < size; i++) {
            int current = queue[i];
            #pragma omp critical
            cout << current << " ";
            for (int j = 0; j < nodes; j++) {
                if (graph[current][j]) {
                    #pragma omp critical
                    if (!visited[j]) {
                        visited[j] = 1;
                        nextQueue[nextSize++] = j;
                    }
                }
            }
        }
        size = nextSize;
        for (int i = 0; i < nextSize; i++)
```

```

        queue[i] = nextQueue[i];
    }
}

// DFS
void dfs(int current) {
    int process = 0;

    #pragma omp critical
    if (!visited[current]) {
        visited[current] = 1;
        process = 1;
        cout << current << " ";
    }

    if (!process) return;
    for (int j = 0; j < nodes; j++) {
        if (graph[current][j] && !visited[j]) {
            #pragma omp task
            dfs(j);
        }
    }
    #pragma omp taskwait
}

int main() {
    int edges, u, v, start;
    cin >> nodes >> edges;
    while (edges--) {
        cin >> u >> v;
        graph[u][v] = graph[v][u] = 1;
    }
    cin >> start;
    bfs(start);
    memset(visited, 0, sizeof(visited));
    cout << "\nDFS: ";
    #pragma omp parallel
    {
        #pragma omp single
        dfs(start);
    }
}

```

```
}  
return 0;  
}
```

```
PS C:\Users\Rutuja Varunkar\Downloads\hpc> g++ -fopenmp P1.cpp -o P1  
PS C:\Users\Rutuja Varunkar\Downloads\hpc> ./P1  
5 4  
0 1  
0 2  
1 3  
2 4  
0  
BFS: 0 2 1 3 4  
DFS: 0 1 2 3 4  
PS C:\Users\Rutuja Varunkar\Downloads\hpc> g++ -fopenmp P1.cpp -o P1  
PS C:\Users\Rutuja Varunkar\Downloads\hpc> ./P1  
5 4  
0 1  
0 2  
1 3  
2 4  
0  
BFS: 0 2 1 3 4  
DFS: 0 2 4 1 3  
PS C:\Users\Rutuja Varunkar\Downloads\hpc> █
```